

Best path: **Blender + Godot 4**.

Why that stack:

- **Blender** is the cleanest way to turn your floor plans into accurate 3D architecture.
- **Godot 4** is free, good for first-person walking, easy to structure building-by-building, and simpler than Unreal for this sort of explorable reconstruction.
- If you want it to stay inside your current website later, you can export the same models to a **Three.js / React Three Fiber** viewer after the Godot version works.

The big rule is this:

Do not build the 3D world from the rendered PNG floor plans alone.

Build it from a **canonical floor data package** for each building and floor, then use the floor plan images only as visual tracing references.

The right pipeline

1. Freeze one authoritative source per floor

For each building and floor, create one structured file that contains:

- outer footprint
- wall lines
- wall thicknesses
- door positions
- stair positions
- window positions
- room labels
- certainty tags: canon / implied / reconstructed
- links to source prompt / book evidence

Use something like:

- `json`
- `yaml`
- or even `svg` + metadata

Example: `archives/ground_floor.json` `hollows/basement.json`
`mains/ground_floor.json`

This matters because if you change a prompt later, your 3D model should still be anchored to one stable plan.

2. Trace each floor plan in Blender

Import each approved floor plan image into Blender as a reference image on the XY plane.

Set:

- **1 Blender unit = 1 metre**
- north always aligned the same way
- one collection per floor
- one collection per building

Then trace:

- exterior walls first
- major corridors
- stairs
- fixed room shells
- then secondary partitions

Do **blockout first**, not detailed modelling.

3. Standardise vertical assumptions

Your floor plans are mostly horizontal. A walkable 3D game needs vertical conventions.

Pick one project-wide vertical standard and stick to it:

- public institutional floor height: around **4.0–4.8 m**
- ordinary upper floors: around **3.2–4.0 m**
- basement / utility floors: **3.0–3.8 m**
- stair riser: around **170 mm**
- stair tread: around **280 mm**
- door height: around **2.1–2.4 m**
- major hall doors can be taller

These are **reconstructed conventions**, not canon, so keep them in a shared config file.

4. Model each building as a stack of aligned floors

For each building:

- Ground floor at $Z = 0$
- Upper floors stacked exactly above
- Basement below
- Stair openings aligned vertically
- Exterior walls aligned unless canon says the upper or lower floor differs

You want:

- Building shell
- Floor slabs
- Walls
- Doors

- **Stairs**
- **Props**
- **Lighting markers** as separate layers or collections

5. Keep “game mesh” and “hero mesh” separate

Make two versions:

- **Game mesh** = simple, low-poly, clean collision
- **Hero mesh** = nicer version for final visuals

At first, only build the **game mesh**.

That means:

- plain walls
- simple floors
- simple door frames
- simple stairs
- no decorative clutter
- no tiny books, pipes, or furniture unless needed for navigation

6. Export to Godot as GLB

Use **.glb** exports from Blender.

One file per building is usually best:

- **archives.glb**
- **hollows.glb**
- **mains.glb**
- **medica.glb**

Or if a building is huge:

- **mains_ground.glb**
- **mains_upper.glb**

How to structure the game

Recommended game structure

Make a **first-person exploration game**, not a traditional “game” first.

Core systems:

- first-person movement

- mouse look
- stairs navigation
- building selector
- floor selector
- map / compass
- hotspot labels
- lore notes panel
- optional noclip mode for debugging

Godot scene structure

Use something like:

```
/project
/scenes
  campus.tscn
  archives.tscn
  hollows.tscn
  mains.tscn
  medica.tscn
  fishery.tscn
  mews.tscn
  masters_hall.tscn
  haven.tscn
  underthing.tscn
/models
  archives.glb
  hollows.glb
  mains.glb
  ...
/data
  buildings.json
  floors/
  rooms/
/scripts
  player/
  ui/
  loading/
  navigation/
```

Campus layout

Have one **master campus scene** with:

- terrain / paths
- major building shells
- entrances

- outdoor navigation

Then either:

- let the player walk into buildings seamlessly, or
- use doorway triggers to load interiors

For big buildings like Mains and the Underthing, doorway loading is cleaner.

What to build first

Build in this order:

1. **Campus blackout**
2. **Archives**
3. **Hollows**
4. **Mains**
5. **Medica**
6. **Fishery / Artificery**
7. **Mews**
8. **Mess**
9. **Masters' Hall**
10. **Haven**
11. **Underthing**

Why:

- campus gives you global alignment
- Archives and Hollows are the cleanest controlled buildings
- Mains and Underthing are the hardest, so don't start there

What “accurate” should mean in practice

You said “as accurate to the generated floor plans as possible.”

That means you should enforce:

Accuracy layer 1

Match the approved floor plans exactly

- wall locations
- room proportions
- stair placement
- entry sequence

Accuracy layer 2

Keep cross-floor alignment consistent

- stairs line up
- halls don't collide
- basements stay under shells
- courtyards align with roof / grate / tree logic

Accuracy layer 3

Mark uncertain things clearly If a room or corridor is reconstructed:

- add a metadata tag
- optionally show a tooltip in-game:
 - canon
 - implied
 - reconstructed

That would make the project much stronger than a normal fan build.

Smart features to add

Must-have

- **Floor overlay toggle**
Press a key to show the 2D floor plan under your feet.
- **Lore hotspot mode**
Click a room to see:
 - room name
 - building
 - floor
 - certainty
 - book evidence
- **Building/floor jump menu**
Jump straight to:
 - Archives Ground
 - Hollows Basement
 - Mains Ground
 - Haven Level 2
 - etc.

Very useful

- **Ghost walls / x-ray mode**
- **Free-fly camera**

- **North compass**
- **Distance ruler**
- **Day / night toggle**
- **Legacy vs current reconstruction toggle**

What Copilot in VS Code should help you build

Copilot is best used for:

- scaffolding the Godot project
- writing data loaders
- building UI panels
- creating debug tools
- validating consistency

It is **not** the best tool for inventing the actual architecture.

Best Copilot task prompt

Paste this into Copilot:

Help me build a Godot 4 first-person exploration project for a canon-constrained 3D reconstruction of the University from the Kingkiller Chronicle.

Project goals:

- Explore campus and interiors in first person
- Walk up and down floors
- Load building models exported from Blender as .glb
- Keep building/floor metadata in JSON
- Support room labels, building labels, and certainty tags: canon, implied, reconstructed
- Add a UI for:
 - building selector
 - floor selector
 - minimap / compass
 - hotspot info panel
 - debug free-fly mode
 - 2D floor-plan overlay toggle
- Add doorway triggers for entering/exiting buildings
- Add optional floor isolation mode to hide other floors
- Add simple save/load position support

Please scaffold:

1. scene structure
2. player controller
3. building loader
4. floor visibility manager
5. hotspot metadata system
6. UI panels

7. JSON schema for buildings/floors/rooms
8. debug tools for checking stair alignment and room overlap

Assume:

- models are authored in Blender
- one Blender unit = one metre
- north must remain consistent across all buildings
- Mains and the Underthing are large and may need chunked loading
- project should be easy to expand building by building

The most important technical rule

Create a file like:

```
{
  "building": "Archives",
  "floor": "Ground",
  "north_up": true,
  "source_plan": "archives_ground_floor_v03.png",
  "certainty": "mixed",
  "entrances": [...],
  "stairs": [...],
  "rooms": [...]
}
```

Then make your game read that data.

That way:

- UI labels are automatic
- floor toggles are automatic
- hotspot text is automatic
- later prompt corrections do not wreck the whole game

If you want browser-based instead of Godot

Use:

- **React Three Fiber**
- **Three.js**
- **GLTF models from Blender**

That is best if you want it to live inside your current website.

But for the fastest path to a walkable explorable build: **Blender + Godot 4 is the better first move.**

The shortest practical plan

This week:

1. freeze campus coordinates
2. make Blender blockout for Archives + Hollows
3. export GLBs
4. make Godot first-person scene
5. add building/floor selector
6. walk around Archives and Hollows first

If you want, I can write you a **full Godot project plan** or a **Blender-to-Godot workflow checklist** next.

You can automate a **lot** of it, but not all of it.

The fastest low-effort setup is:

AI for structure, Blender for geometry, Godot for walking, scripts for glue.

Best low-effort pipeline

1. Make one structured source of truth

Stop building from loose prompts and PNGs.

Have AI turn each approved plan into a simple JSON like:

```
{
  "building": "Archives",
  "floor": "Ground",
  "footprint": [...],
  "walls": [...],
  "doors": [...],
  "stairs": [...],
  "rooms": [...],
  "windows": [...],
  "certainty": "mixed"
}
```

Do this for every building and floor.

Why:

- then everything downstream can be automated
- Blender import can be scripted
- Godot labels and navigation can be automatic
- corrections only need one change

2. Use AI to convert prompt packs into floor JSON

Use Copilot or Codex to build a parser workflow:

- input: approved prompt + approved image + any notes
- output: `building_floor.json`

Have AI generate the first draft, then you quickly check it.

This is much faster than hand-modelling from scratch.

3. Auto-generate Blender blockouts from JSON

This is the biggest time saver.

Have a Blender Python script that:

- reads the floor JSON
- creates wall curves or meshes
- extrudes walls to standard heights
- cuts doors
- places stairs as simple block meshes
- places floor slabs
- names collections by building/floor/room

Then one click gives you a walkable blockout.

4. Auto-stack floors into buildings

A second script should:

- load all floor JSON files for a building
- place them at the right Z heights
- align stairs
- create building shells
- export one `.glb` per building

That gets you from “2D plans” to “3D building” with very little manual effort.

5. Auto-build Godot scenes from metadata

Then use Godot scripts to:

- load `.glb` buildings

- spawn them in campus positions
- generate labels from JSON
- create hotspot info panels
- create floor toggles
- create building jump menu
- create certainty overlay: canon / implied / reconstructed

So instead of hand-placing everything in Godot, the game becomes mostly **data-driven**.

What AI should do vs what you should still do

Let AI do

- prompt cleanup
- plan-to-JSON drafting
- Blender script generation
- Godot scene scaffolding
- hotspot metadata
- room labels
- consistency checking
- file naming / structure
- batch export tools

You still do

- approve the canonical floor data
- fix weird stair geometry
- check proportions
- decide heights where canon is silent
- visually inspect walkability
- reject bad AI assumptions

That's the real balance.

Least-effort automation stack

Option A — best practical stack

- VS Code + Copilot / Codex
- Python scripts
- Blender
- Godot 4

This is the best mix of power and realism.

Option B — even lower effort, more limited

- keep it in the browser
- use **Three.js / React Three Fiber**
- load simplified generated 3D directly on the website

This is easier to share, but harder to make into a good first-person explorable world quickly.

The automation architecture I'd use

Folder structure

```
/university-project
/canon
  /buildings
    archives_ground.json
    archives_level2.json
    hollows_ground.json
    ...
/images
/approved
/blender
  import_floorplans.py
  build_building.py
  export_glb.py
/models
  archives.glb
  hollows.glb
/godot
/data
/scenes
/scripts
```

Flow

1. approved prompt/image
2. AI drafts floor JSON
3. you approve JSON
4. Blender script builds floor
5. Blender script stacks building
6. export GLB
7. Godot auto-loads building
8. UI auto-loads labels and hotspots

Best exact automation targets

Automate first

- Archives
- Hollows
- Medica
- Masters' Hall
- Mews
- Mess

These are more regular and easier to block out.

Automate later

- Mains
- Underthing
- Haven interiors

These are much messier and need more human checking.

Specific things Copilot should build for you

Ask Copilot to build these in order:

1. Floor JSON schema

A strict schema for:

- walls
- doors
- stairs
- rooms
- windows
- labels
- certainty tags

2. Blender importer

A script that:

- reads JSON
- draws wall splines

- extrudes them
- inserts doors/stairs
- creates collections

3. Building stacker

A script that:

- combines multiple floors
- places them vertically
- validates stair alignment

4. Godot loader

A script that:

- loads all GLBs
- places them on campus
- adds hotspot metadata
- handles floor toggles

5. Validation tool

A Python checker that flags:

- stairs that don't align
- rooms outside shell
- door collisions
- missing metadata
- floor naming mismatches

That validation step will save you a lot of pain.

Best Copilot prompt for this

Paste this into VS Code Copilot:

I want to automate a data-driven pipeline for turning approved Kingkiller Chronicle floor plans into a walkable 3D Godot project.

Build this in stages:

1. Define a JSON schema for a building floor plan with:

- building
- floor
- footprint

- walls
- doors
- stairs
- windows
- rooms
- labels
- certainty tags (canon, implied, reconstructed)
- metadata links to source prompt/image

2. Write a Python tool that reads those JSON files and validates:

- closed wall loops
- stairs alignment between floors
- room containment inside footprint
- duplicate IDs
- missing labels

3. Write a Blender Python script that:

- imports one floor JSON
- creates floor slab
- extrudes walls
- cuts door openings
- places simple stairs
- creates collections by building/floor
- names meshes cleanly

4. Write a second Blender script that:

- stacks all floors for a building vertically
- aligns stairs
- exports one GLB per building

5. Write a Godot 4 loader that:

- loads building GLBs
- places them on a campus map
- supports first-person walking
- supports floor visibility toggles
- supports hotspot metadata panels
- supports building/floor jump menu

6. Keep everything modular and easy to expand building-by-building.

Important:

- Prefer simple walkable blockouts over pretty meshes
 - Assume one Blender unit = one metre
 - Assume north must stay consistent
 - Preserve uncertainty tags in metadata
 - Do not hardcode room names outside the JSON
-

The absolute least-effort version

If you want the **fastest possible** route, do this:

1. freeze campus positions
2. make AI produce JSON for each floor
3. use Blender scripts to auto-extrude all floors
4. export to Godot
5. walk around simple blockouts first
6. only add detail later

That gives you a playable 3D University much faster than trying to hand-model beautiful rooms first.

Honest limit

AI can get you to **70–85% of the pipeline** much faster.

The bits that still need you:

- approving canon decisions
- fixing weird geometry
- deciding heights and unclear joins
- choosing between contradictory reconstructions

That's normal.

If you want, I can write the **exact JSON schema** for your building/floor system next.

Transforming these "canon-constrained" reconstructions into a walkable 3D environment is a massive undertaking, but honestly, it's the ultimate dream for any *Kingkiller* fan. Since you have the floor plans, you have the "blueprint" layer finished. Now you need to move into the ****Digital Stonework**** phase.

Here is the roadmap to turning these 2D plans into a 1:1 virtual University.

1. Pick Your Engine

For a project focusing on atmospheric lighting and interior "vibes," you have two main contenders:

* ****Unreal Engine 5 (UE5):**** Best for ****Atmosphere****. Its "Lumen" lighting system is perfect for those unwavering sympathy lamps and the silvery moonlight on the Masters' Hall. It handles massive architectural detail very well.

* ****Unity:**** Best for ****Accessibility****. If you're new to coding, Unity has a massive library of "First Person Controller" assets that will get you walking in minutes.

2. The "Blueprint to Bricks" Workflow

You can't just drop a 2D image into a game and expect it to have collisions. You need to ****extrude**** the geometry.

Step A: Reference Planes

Import the floor plan images as a "Plane" in a 3D modeling tool like **Blender** or directly into the engine using a tool like **ProBuilder** (Unity) or **Modeling Mode** (Unreal). Scale the plane so the 10m scale bar on the image matches 10 units in the engine.

Step B: Blocking Out (Greyboxing)

Trace the walls. Don't worry about textures yet. Just create simple boxes where the walls are.

- * **Archives:** Focus on the thick, windowless outer shell first.

- * **Mains:** This will be your hardest task because of the "lichen-like" growth. You'll need to vary the heights of different "absorbed" building sections.

3. Mastering Verticality

Accuracy depends on how the floors stack.

- * **Vertical Alignment:** Create a "Master Stairwell" object. In the Archives, align the **Southeast Stairwell** across all 6 floors. If the stairs don't line up perfectly due to AI generation quirks, you'll have to "nudge" the floor plans until the stairs create a continuous path.

- * **The Underthing:** This should be its own "level" or a massive basement. Use the "Drainage Grate" in the **Hidden Apple-Tree Courtyard** of Mains as your vertical anchor.

4. Canon-Accurate Lighting

Lighting is what makes the University feel real. You'll want to use **Baked Lighting** for the Archives and **Dynamic Lighting** for areas with "magic."

- * **Sympathy Lamps:** Place small, point-light sources with a slightly reddish/warm hue in the Archives and Tomes.

- * **The "Circle of Light":** In the Admissions Theatre, use a **Spotlight** with a sharp falloff to create that interrogation feel.

- * **The Stacks:** Keep the ambient light at near-zero. Force the player to use a "Hand Lamp" (a point light attached to the player's camera) to see the shelving labels.

5. Interaction and "Feel"

Walking is easy; interacting is where the lore comes alive.

| Feature | Implementation |

|---|---|

| **Heavy Doors** | Use "Physics Constraints." The stone doors of the Archives should feel heavy and slow to open. |

| **The "Sticky" Door** | For Elodin's Naming Hall, write a simple script that requires the player to "Interact" 3 times before the door successfully opens. |

| **Lockpicking** | In Hollows, you can implement a simple UI mini-game to simulate Kvothe picking the two locks to get into the observation area. |

6. Dealing with "The Labyrinth" (Mains)

Since Mains is an "acre and a half" of nonsense, it will be your biggest performance hurdle.

- * **Level Streaming:** Only load the wing the player is currently in.

- * **NavMesh:** You'll need to generate a "Navigation Mesh" so the engine knows where the floor is. Because Mains has "stairs to nowhere," you might find your character getting stuck in "bricked-off" rooms. You'll have to manually "clip" these areas so the player can't phase through the walls into the void.

Pro-Tip: The "Blender" Shortcut

If you want to be *truly* accurate, I recommend using **Blender** to build the 3D shells first. Trace the floor plans, "E" (Extrude) the walls up to a standard 3-4 meter height, and then export that as an .FBX file into your game engine.

****Would you like to start with a specific building? I can break down the "Room Heights" and "Ceiling Logic" for the Archives or Mains to help you stack them!****

To do this with "least effort," you have to stop thinking like an architect and start thinking like a ****systems integrator****. You want to use a pipeline where one AI interprets the plan and another AI builds the geometry.

Here is the "Lazy Path" to a walkable University, ranked by speed and automation.

1. AI Image-to-3D Extrusion (The 5-Minute Method)

The fastest way to get walls is to use a dedicated ****Floor Plan to 3D**** service. These are usually designed for real estate, but they work for "Tomes" or "The Medica" if the lines are clean.

*** **Tools:**** ****CubiCasa****, ****Planner 5D****, or ****Canvas (by Occipital)****.

*** **Process:**** 1. Upload the generated floor plan image.

2. The AI identifies "walls," "doors," and "openings."

3. Export as an .OBJ or .FBX.

*** **Downside:**** These tools struggle with the "Underthing" or "Mains" because they expect 90-degree residential walls. They might try to turn a "Runic Grate" into a window.

2. The "Blender Scripting" Shortcut (The Pro-Automation Path)

Since you already have clean black-and-white plans, you can use ****ChatGPT or me (Gemini)**** to write a ****Python script for Blender**** that automates the extrusion.

*** **The Workflow:****

1. ****Image Processing:**** Use a simple AI tool to "Vectorize" your floor plan (convert it to an .SVG).

2. ****AI Scripting:**** Ask an AI: ****"Write a Blender Python script that imports an SVG, selects all paths, and extrudes them by 4 meters on the Z-axis."****

3. ****Result:**** You get a full 3D shell of the floor in seconds without manually dragging a single wall.

3. Generative AI for Furniture & Props (Populating the Rooms)

You don't want to spend hours modeling "300 Archive Shelves" or "Hemme's Chair." Use ****Text-to-3D**** or ****Image-to-3D**** generative models.

*** **Tools:**** ****Meshy.ai****, ****Rodin (Deemos)****, or ****Luma AI Genie****.

*** **Process:**** 1. Take a screenshot of a specific room label (e.g., "Bursar's Counter").

2. Feed it into the AI with the prompt: ****"A 17th-century heavy oak bursar's counting desk, dark wood, realistic, game-ready."****

3. Download the asset and drop it into your engine.

4. Total Automation: AI Game Engines

There are emerging platforms where you can "describe" the floor plan and they build the environment for you.

*** **Tool:**** ****Promethean AI****. This is an AI that works **inside** Unreal Engine. You can tell it: ****"This is a library room. Populate it with bookshelves and long tables consistent with the layout."**** It will automatically search your asset library and place items in the "Tomes" room based on the floor plan's logic.

*** **Tool:**** ****Hume AI or Convai****. For the "Characters," you can give an AI NPC (like Puppet or Master Kilvin) the **Kingkiller** lore, and they will talk to you naturally as you walk through your built environment.

5. The "Minimum Viable Product" Pipeline

If you want to be walking through the Archives by this evening, do this:

1. ****Vectorize:**** Use a free online "Image to SVG" converter on your floor plans.

2. **Extrude:** Use a Python script in Blender (I can provide the code) to pop those walls into 3D.
 3. **Export:** Send the whole thing to **Roblox Studio** or **Unity**.
 - * **Why Roblox?** Because the "First Person Controller" and "Multiplayer Server" are built-in. You just hit "Publish" and you can walk through the Stacks with your friends immediately.
 4. **Texture:** Use an AI texture generator (like **Polycam** or **Leonardo.ai**) to create a "Seamless Grey Stone" texture and apply it to every wall at once.
- ### Which building should we automate first?
- If you give me one building to focus on, I can provide a **Python script** you can paste into Blender right now to turn that 2D image into 3D walls. Which one do you want to start with?

Yes. Here's the cleanest breakdown.

1. Things I can do for you right now

1. Write the **exact JSON schema** for your building / floor / room data model.
2. Write **example JSON files** for real parts of your project, like:
 - `archives_ground.json`
 - `hollows_basement.json`
 - `mains_ground.json`
3. Turn your approved prompts and floor-plan logic into **structured canonical data** you can actually build from.
4. Write a **shared reconstruction config** for non-canon defaults, like:
 - floor heights
 - stair rise / tread
 - wall thickness defaults
 - door heights
 - window defaults
 - certainty tags
5. Write a **Blender Python importer** that:
 - reads floor JSON
 - creates floor slabs
 - extrudes walls
 - places simple doors and stairs
 - organises collections by building and floor
6. Write a **Blender building stacker** that:
 - loads all floors for one building
 - stacks them vertically

- aligns stairs
 - exports **.glb**
7. Write a **validation script** that checks:
 - stairs line up between floors
 - rooms stay inside shells
 - duplicate IDs
 - missing labels
 - bad north alignment
 - contradictions in metadata
 8. Write a **Godot 4 project scaffold** for:
 - first-person walking
 - floor toggles
 - building selector
 - room hotspots
 - lore info panel
 - free-fly mode
 - save/load position
 9. Write the **Godot scripts** for loading building metadata and **.glb** files.
 10. Write the **UI spec** for the exploration tool or game.
 11. Write the **folder structure** for the whole project so it stays sane.
 12. Write the **Copilot / Codex prompts** to get VS Code doing most of the implementation work.
 13. Audit your current prompts, floor plans, and website logic for **canon drift and internal contradictions**.
 14. Create a **master backlog** of what still needs to be generated, modeled, checked, or replaced.
 15. Turn your current atlas / website into a more useful **production tracker spec**.
 16. Write the **Three.js / React Three Fiber** architecture if you want browser-based walkthroughs later.
 17. Help you define a **minimum viable version** so you do not drown in scope.
 18. Generate more **2D prompt packs** for any missing buildings / rooms / floors.
 19. Generate more **reference images** when you want them.

20. Review output from other tools and tell you **what is wrong, what is good, and what to fix next**.
21. Help you design the **certainty overlay system** so users can see canon / implied / reconstructed in-game.
22. Write a **step-by-step implementation plan** for one week, one month, or phased milestones.
23. Write **README files, build docs, naming conventions, and project standards**.
24. Help you convert this from “cool fan project” into a **repeatable pipeline**.

2. Things I cannot do directly here, but another AI tool can do

1. Open Blender on your machine and actually click, trace, and save the models.
2. Open Godot on your machine and run, test, and debug the project live.
3. Open VS Code locally and edit your repo files directly on your computer.
4. Use **Codex / Copilot in VS Code** to make many-file changes automatically inside your working project.
5. Execute local terminal commands on your actual machine, install dependencies, and fix environment issues in place.
6. Import floor plans into Blender and visually adjust bad geometry by hand.
7. Export **.glb** files from your own Blender installation.
8. Build and launch the 3D walkthrough locally in Godot.
9. Inspect local rendering problems, framerate issues, collision bugs, or shader issues on your hardware.
10. Use agentic coding tools to repeatedly modify, run, test, and patch your codebase in a tight loop.
11. Batch-process dozens of your local files directly unless you upload them here.
12. Integrate with your exact file paths, local plugins, and VS Code workspace settings without access to them.

13. Do visual in-editor tasks like:

- repositioning stairs
- checking ceiling clearance
- fixing navigation meshes
- adjusting collisions
- tweaking camera feel

14. Turn your current files into a full playable game automatically without some external execution environment.

15. Handle long-running automated pipelines on your computer while you are away, unless another tool/service is doing that.

3. Things only a human can actually do

1. Decide which reconstruction choices you are willing to **canonically accept** when the books are ambiguous.
2. Choose whether a result is “good enough” artistically, emotionally, and aesthetically.
3. Judge whether the project still feels like **Rothfuss’s University** rather than a technically correct but soulless layout.
4. Make final calls when two plausible reconstructions conflict.
5. Notice when something “feels wrong” even if all the metadata looks correct.
6. Decide how much uncertainty you want to expose versus smooth over for users.
7. Prioritise your own time, energy, and scope so the project stays sustainable.
8. Decide what version of the project matters most:
 - pure scholarship
 - explorable fan experience
 - game
 - web atlas
 - all of the above later
9. Approve or reject AI-generated geometry, layouts, naming, and atmosphere.
10. Catch subtle lore tone issues that are not just factual but interpretive.
11. Decide when to stop polishing one building and move on.
12. Live with the creative consequences of the choices.

Blunt answer

I can do most of the planning, structuring, schema design, prompt writing, code writing, validation logic, and pipeline design right now.

I cannot directly operate Blender, Godot, or your local VS Code.

You still have to be the canon judge and final artistic authority.

Best immediate next thing for me to do

The highest-value thing I can do next is:

1. write your **exact floor JSON schema**
2. write **one complete example** for `archives_ground.json`
3. write the matching **Blender importer script skeleton**
4. write the matching **Godot loader schema**

That would turn all this from ideas into a real build pipeline.

To turn these plans into a 3D game with "least effort," we need to shift from **image generation** to **data generation**. As an AI, I can't reach into your computer and click "Extrude," but I can provide the "digital DNA" that makes those clicks automatic.

Here is exactly how we divide the labor for this project.

1. What I Can Do For You Right Now

I act as your **Architectural Data Engine**. I can generate the logical blueprints that other software reads.

1. **Generate Structured JSON/YAML Data:** I can convert any floor plan we've designed into a machine-readable format. This "data package" tells a game engine exactly where a wall starts and ends so you don't have to trace them manually.
2. **Write Blender Python Scripts:** I can write custom scripts for Blender that you simply "Run" to automatically extrude walls, cut door openings, and place floor slabs based on the data I generate.
3. **Code Godot 4 Systems:** I can provide the GDScript for your first-person controller, floor-toggle menus, and "hotspot" systems that display lore notes when you walk into a specific room.
4. **Enforce Canon Logic:** I can audit your 3D progress. If you describe a room to me, I can tell you if it violates the *Kingkiller* constraints (e.g., "You can't have a window there; the Archives are windowless").

5. **Generate Procedural Textures:** I can write "Shader Code" (GLSL) for Godot to create the "all-meshed stone" or "shadowy cool" lighting effects described in the books.

2. What Other AI Tools (Copilot/Codex/Plugins) Can Do

These tools act as your **Integrated Laborers**. They live inside your "workshop" (IDE) and handle the grunt work of construction.

1. **Project Scaffolding:** VS Code Copilot can generate the entire folder structure for your Godot project in seconds, including the `/scenes`, `/scripts`, and `/models` directories.

2. **Real-Time Autocomplete:** As you type code to make the "Archives Stone Doors" open, Copilot will suggest the rest of the logic based on how you've written previous doors.
3. **Automated Unit Testing:** AI can write "QA bots" that virtually walk through your building to check for "clipping" (walking through walls) or "holes" in the geometry.
4. **Asset Management:** Tools like *Promethean AI* can take my "Tomes" plan and automatically scatter "Academic Props" (books, inkwells, chairs) into the room based on the labels.
5. **Refactoring:** If you decide to change the building scale from 1 unit = 1m to 1 unit = 2m, these tools can update every script in your project at once.

3. What Only a Human Can Do

You are the **Master of the University**. You provide the soul, the quality control, and the final "vibe" check.

1. **Curatorial Approval:** AI might place a stairwell that technically works but *feels* wrong for a 5th-century stone monolith. You make the final call on what is "Canon-Accurate."
2. **Atmospheric Fine-Tuning:** AI can set a "light level," but only a human can adjust the fog, shadows, and reddish hue of a sympathy lamp until it feels "eerily quiet."
3. **Level Design "Flow":** You decide if the distance between the Archives and Hollows is too frustrating to walk in a game. You manage the "fun" factor.
4. **Complex Spatial Solving:** When the AI generates a basement that accidentally overlaps with the Underthing, you are the one who performs the "surgical" move to fix the geometry.
5. **Intentional Personality:** Adding the "messy" details—like a half-eaten apple on a stone bench in the hidden courtyard or the way a door "sticks"—requires human storytelling intent.

Would you like me to start by generating the JSON data and a Blender script for the "Archives Ground Floor" so you can try a one-click 3D extrusion?

Turning 2D images into 3D models with AI (ChatGPT + Blender Tutorial)

This video provides a step-by-step walkthrough of the exact Python scripting workflow I recommended to automate 3D modeling from your 2D plans.

YouTube video views will be stored in your YouTube History, and your data will be stored and used by YouTube according to its Terms of Service

Since ChatGPT is handling the **JSON schemas**, **Python scripts**, and **Godot scaffolding**, I will handle the **Vertical Spatial Calibration** and the **Environmental Art Direction**.

While ChatGPT focuses on the "what" and the "how," I will provide the **"Y-Axis Master Data"** and the **"Atmospheric Shader Logic."** These are the missing links between a 2D floor plan and a 3D world that feels like the University.

1. Vertical Z-Axis Calibration Master Table

Your floor plans are 2D (X and Z in game coordinates). For a walkable Godot project, you need to know exactly where to place each floor on the **Y-axis** (height) so stairs line up and the "Underthing" doesn't collide with the Archives foundations.

Building	Level	Base Y-Offset (m)	Floor-to-Ceiling (m)	Role in Game Stack
Archives	Level -3 (Underthing)	-12.0m	4.0m	Deepest reachable segment;
Archives	Level -2 (Crawl)	-8.0m	3.0m	Ventilation & hidden access crawlspace.
Archives	Level -1 (Foundation)	-4.5m	3.5m	Archives structural base.
Archives	Ground Floor	0.0m	8.0m	Tomes (Double height for volume).

Belows.

Archives Level -3 (Underthing) -12.0m | 4.0m | Deepest reachable segment;

Belows.

Archives Level -2 (Crawl) -8.0m | 3.0m | Ventilation & hidden access crawlspace.

Archives Level -1 (Foundation) -4.5m | 3.5m | Archives structural base.

Archives Ground Floor 0.0m | 8.0m | Tomes (Double height for volume).

| **Archives** | **Levels 2-5** | +8.0m to +20.0m | 4.0m each | Standard Stacks-city neighborhoods. |

| **Archives** | **Level 6 (Top)** | +24.0m | 4.5m | Top floor; leads to roof housing. |

| **Hollows** | **Basement** | -4.0m | 3.5m | Bursar's counting room & vault. |

| **Hollows** | **Ground Floor** | 0.0m | 5.0m | Admissions Theatre (Raised stage logic). |

| **Hollows** | **Second Floor** | +5.0m | 4.0m | Master Offices; Clerk's Room. |

| **Mains** | **Underthing Level** | -4.0m | 3.5m | Runic grate access from courtyard. |

| **Mains** | **Ground Floor** | 0.0m | 4.5m - 9.0m | Variable heights due to "fused" buildings. |

> **Note:** These are **reconstructed standards**. Using these unified heights in your JSON will ensure that if you walk out of the Archives Ground Floor and across to Hollows, your "eye level" remains consistent.

>

2. Underthing Topology: Node Connectivity Map

Building the Underthing as a "maze" is hard. Instead of just modeling a tunnel, you should build it as a **Node-based network**. Here is the logical connectivity you should use to link the separate buildings in Godot:

* **Node A (Archives Crawl):** A narrow "belly-crawl" ventilation shaft.

* **Connects to:** Archives Ground Floor (Hidden panel) AND Node B.

* **Node B (The Tunnels):** Standard stone tunnels with exposed steam pipes.

* **Connects to:** Node A, Node C, and Node D.

* **Node C (Cricklet):** Long, low, and watery. Hazardous floor.

* **Connects to:** Node B.

* **Node D (The Woods):** Large ruined chamber with timber supports.

* **Connects to:** Node B AND Node E.

* **Node E (The Runic Grate):** Vertical transition (Ladder/Climb).

* **Connects to:** Node D AND Mains Hidden Apple-Tree Courtyard.

3. Atmospheric "Sympathy" Shader Logic

To make the Archives feel "windowless but lit," do not use standard "Sun" light in Godot. You need a **Global Illumination (GI)** strategy that emphasizes the "unwavering reddish light" of sympathy lamps.

The "Archives Lamp" Shader Specs:

* **Albedo:** Base color #FF7F50 (Coral/Warm Red).

* **Emission:** Set to High (2.0 - 5.0 strength).

* **Volumetric Fog:** In Godot 4, enable "Volumetric Fog." Set the Density to 0.02. This will make the "dust" mentioned in the books catch the light from the sympathy lamps as you walk past.

* **Shadows:** Use **Soft Shadows**. In the Stacks, light should not travel far. Set the Range of your point lights to only 8-10 meters.

4. Technical Quality Control Backlog

While ChatGPT is writing scripts, I have identified these **Logical Contradictions** in the current plan that you need to resolve before the 3D model is built:

1. **The Stacks/Tomes Boundary:** In the 2D plan, the transition from "Well-lit Tomes" to "Dark Stacks" is a single door. In 3D, you will need a **Light Probe** at that doorway to prevent the bright light of Tomes from "bleeding" into the dark Stacks through the wall geometry.

2. **Mains Roof Patchwork:** Since Mains is many fused buildings, your 3D model cannot have a flat floor. You should add **1-3 steps (30cm to 50cm)** of elevation change at every

"Building Seam" in the corridors to represent the height differences between the old engulfed buildings.

3. **The Four-Plate Door:** This is canonically on a "lower level." Your current 2D stack needs to ensure that the Archives stairwell reaches at least **Level -1** before the player can find the hallway leading to the Four-Plate door.

5. Visual Art Direction: "All-Meshed Stone"

I am generating a **Material Reference Sheet** for you. This image shows the specific texture and lighting "target" for your Godot shaders: the grey, heavy stone of the Archives vs. the warm, eerie glow of a single sympathy lamp.

What would you like to do next?

1. I can generate the **Sygaldry/Rune Assets** (visual references) for the drainage grate.
2. I can write the **"Soundscape Brief"** (Ambiance, footsteps on stone, muffled echoes) for the game engine.
3. I can provide the **detailed X/Z coordinates** for the "Master Campus Anchor Point" so all buildings align.

This phase bridges the gap between your technical blueprints and the immersive "feel" of the University. Here is the data for your vertical calibration, runic assets, and auditory environment.

1. Master Campus Anchor Point (Global Coordinates)

To ensure every building fits perfectly in Godot, we use a **Global Origin (0,0,0)**. All buildings must be placed relative to this point.

Origin (0,0,0): The geometric center of the **Archives Ground Floor**.

Coordinate System: X = East/West, Y = Height, Z = North/South.

Scale: 1 unit = 1 meter.

Global Offset Table

Building	Center (X, Z)	Rotation (Y-axis)	Base Elevation (Y)	Note
----------	---------------	-------------------	--------------------	------

--	--	--	--	--

Archives	(0, 0)	0°	0.0m	The central anchor.
-----------------	--------	----	------	---------------------

Hollows	(-85, -60)	15°	-1.5m	Sited "down and left" from Archives.
----------------	------------	-----	-------	--------------------------------------

Mains	(110, 45)	-5°	0.0m	Sprawling east; fused geometry.
--------------	-----------	-----	------	---------------------------------

Masters' Hall	(-110, 40)	0°	1.0m	Separate residential building.
----------------------	------------	----	------	--------------------------------

The Mews	(0, 140)	0°	0.5m	North of the Archives.
-----------------	----------	----	------	------------------------

Medica	(180, -20)	25°	0.0m	Sited toward the busy road.
---------------	------------	-----	------	-----------------------------

2. Sygaldry/Rune Assets: The Binding Grate

In canon, the drainage grate in the hidden courtyard is older than the building and carries a specific runic binding: **Ule and Doch**.

The Runes: **Ule** and **Doch** are the fundamental runes for binding things together. On the grate, they are etched to prevent it from being easily lifted without the correct "Alar" or mechanical key.

Visual Target: The runes should be angular, deep-set, and look as though they were "burned" into the iron rather than stamped.

[invalid URL removed]

3. University Soundscape Brief

Sound defines the "pressure" of the University buildings. Use these profiles for your Godot audio buses.

A. The Archives (The "Vacuum" Bus)

* **Ambiance:** A thick, heavy silence. Use a low-pass filter to muffle all high-frequency sounds. A very faint, nearly inaudible "white noise" hum represents the cool, dry air moving through ventilation.

* **Foley:** Footsteps on stone should be crisp but have zero reverb—the books and shelves act as acoustic dampeners.

* **Specifics:** The muffled "thump-slide" of a heavy stone door opening.

B. Mains (The "Echo" Bus)

* **Ambiance:** Distant, muffled echoes of voices or shifting chairs from other floors.

Because of the labyrinthine turns, sound should feel "decoupled" from its source.

* **Foley:** High reverb. Footsteps should ring out, sounding as though they are coming from a different corridor than the one you are in.

* **Specifics:** The scratching of chalk on slate; the faint "whoosh" of milkweed seeds in Elodin's lecture hall.

C. The Underthing (The "Industrial" Bus)

* **Ambiance:** A deep, tectonic vibration. The rhythmic "thrum-clang" of steam pipes heating up and cooling down.

* **Foley:** Wet splashes in **Cricklet**;; hollow metallic "pings" when walking over iron grates.

* **Specifics:** The distinct "red-hot" hiss when standing near the exposed steam pipe zones.

D. Hollows (The "Dignified" Bus)

* **Ambiance:** Total stillness occasionally broken by the scratching of a quill in **Ledgers and Lists**.

* **Foley:** Soft echoes in the vaulted entryway. Footsteps on the marble floor should have a sharp "clack" with a clean, high-frequency tail.

Next Implementation Step: The "Json-to-Godot" Parser

With these coordinates and assets, you are ready to begin the **Data-to-Scene** automation. Would you like me to generate the first **Archives_Ground.json** data file using these X/Z coordinates?

Here is the first authoritative data file for your pipeline: archives_ground.json.

This file uses the **Master Campus Anchor Point (0,0,0)** and the specific room logic established in our previous reconstructions. It is formatted to be easily parsed by the Blender Python script or the Godot loader.

archives_ground.json

```json

```
{
 "building": "Archives",
 "floor": "Ground",
 "y_offset": 0.0,
 "floor_height": 8.0,
 "north_alignment": 0,
 "metadata": {
 "certainty": "canon-constrained",
 "source": "NOTW Ch 36, Ch 38",
 "description": "The windowless grey stone monolith. Public threshold logic."
 }
}
```

```

},
"shell": {
 "type": "box",
 "dimensions": {"x": 40.0, "z": 40.0},
 "origin": {"x": 0.0, "z": 0.0}
},
"entrances": [
 {
 "id": "main_north_gate",
 "name": "The Stone Doors",
 "pos": {"x": 0.0, "z": 20.0},
 "type": "stone_double",
 "inscription": "Vorfelan Rhinata Morie",
 "status": "public_threshold"
 }
],
"rooms": [
 {
 "id": "antechamber",
 "name": "Antechamber",
 "bounds": {"min": {"x": -5.0, "z": 12.0}, "max": {"x": 5.0, "z": 20.0}},
 "lighting": "reddish_sympathy",
 "fittings": ["ledger_counter", "scriv_station"]
 },
 {
 "id": "tomes",
 "name": "Tomes",
 "bounds": {"min": {"x": -15.0, "z": -15.0}, "max": {"x": 15.0, "z": 10.0}},
 "ceiling_height": 7.5,
 "lighting": "bright_sympathy",
 "fittings": ["long_reading_tables", "public_shelving"]
 },
 {
 "id": "scrivs_only",
 "name": "Scriv's Office (West Wing)",
 "bounds": {"min": {"x": -19.0, "z": -10.0}, "max": {"x": -16.0, "z": 15.0}},
 "access": "restricted_scriv",
 "fittings": ["transcription_desks", "copying_tools"]
 }
],
"navigation": {
 "stairs": [
 {
 "id": "southeast_stair_up",
 "pos": {"x": 12.0, "z": -12.0},
 "target": "level_2",
 "direction": "up",
 "type": "stone_spiral"
 }
]
}

```

```

 },
 {
 "id": "southeast_stair_down",
 "pos": {"x": 14.0, "z": -12.0},
 "target": "underthing_level_1",
 "direction": "down",
 "type": "stone_spiral"
 }
],
 "corridors": [
 {
 "id": "stacks_initial_access",
 "name": "East Stacks Entrance",
 "start": {"x": 5.0, "z": 15.0},
 "end": {"x": 18.0, "z": 15.0},
 "type": "restricted_arcanum"
 }
]
}
...

```

### ### Implementation Strategy

#### ##### 1. The Blender "One-Click" Script

You can now take this JSON and feed it into a Python script in Blender. The script will:

- \* Create a plane from -20 to 20 on both axes for the **\*\*Shell\*\***.
- \* Extrude walls to 8.0m (Tomes' volume).
- \* Cut a boolean hole at (0, 20) for the **\*\*Stone Doors\*\***.
- \* Place a placeholder mesh for the **\*\*Southeast Stairwell\*\***.

#### ##### 2. The Godot Collision Layer

When you import the resulting .glb into Godot, the rooms data in the JSON allows you to set up **\*\*Area3D\*\*** nodes.

\* When the player enters the antechamber area, the engine triggers the **\*\*"Vacuum" Audio Bus\*\***.

\* When the player enters tomes, the engine shifts the **\*\*Global Illumination\*\*** to bright\_sympathy.

#### ##### 3. Scaling for the Underthing

Notice the southeast\_stair\_down leads to underthing\_level\_1. By having this JSON, Godot knows exactly where to "spawn" the Underthing level so that when the player walks down the stairs, they physically arrive at the correct X/Z coordinates under the building.